Uso de JavaScript (III) Llamadas AJAX mediante REST y JSON

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

Restricciones arquitectónicas de REST

Arquitectura cliente-servidor

Sin estado

Cacheable

Modelo en capas

Interfaz uniforme

- Uso de URIs para identificar recursos
- HATEOAS
- Métodos HTTP estándar: GET, POST, PUT, DELETE, PATCH

Modelo de comunicación REST

GET /employees/1

HTTP 200 OK

{
"name": "Juan Sánchez",
"salary": 500,
"departmentId": 23
}

Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

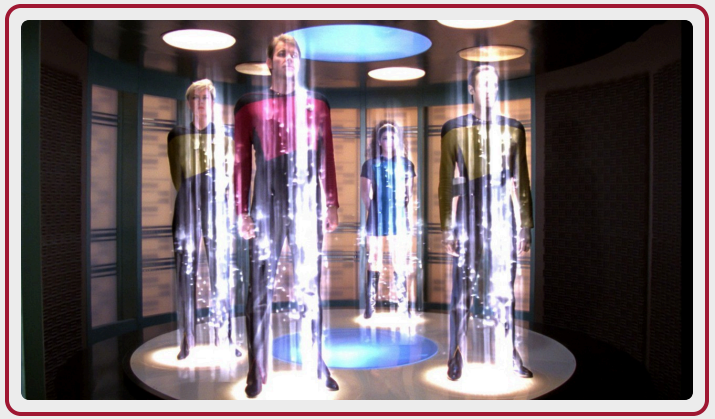
Ejemplos

Módulos de comunicación con API

Serialización

…es el proceso de codificación de objetos o estructuras de datos a un formato que permita ser almacenados (por ejemplo, en un fichero o un buffer de memoria) o transmitidos (por ejemplo, a través de una conexión de red), y posteriormente reconstruidos (posiblemente, en un sistema informático diferente).

Al proceso contrario (la reconstrucción de los objetos) se le denomina “deserialización”.



Serialización para transmisión de datos

serialización

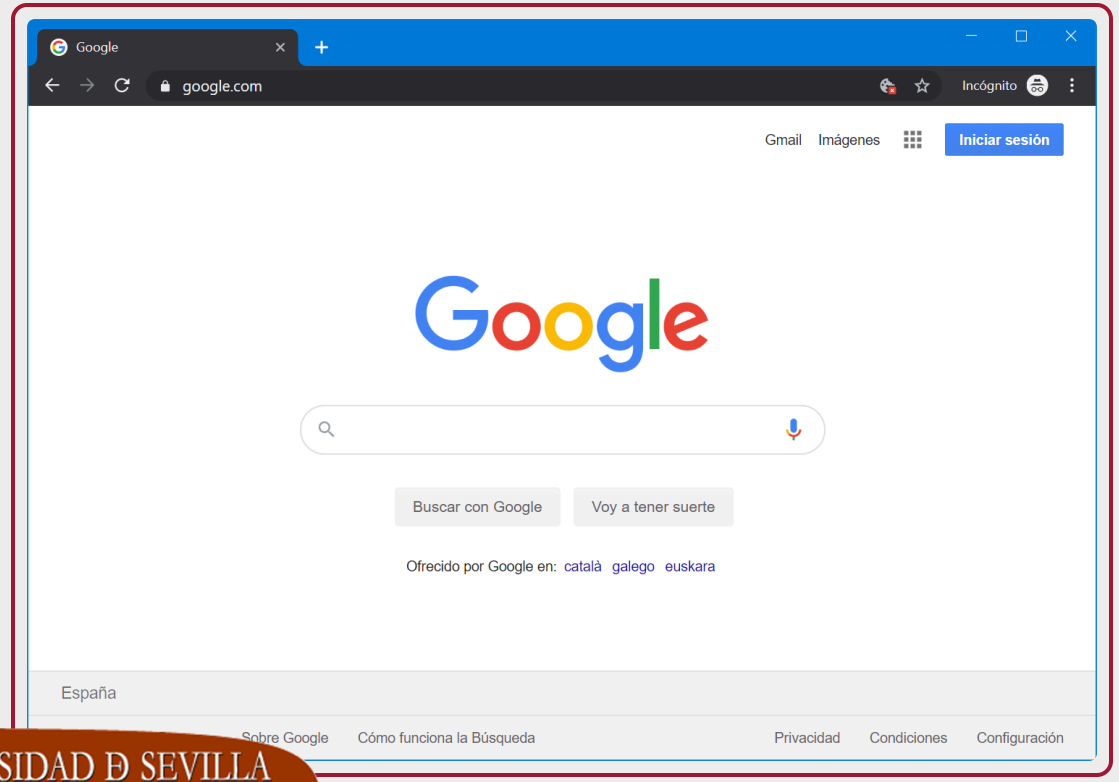
deserialización

JavaScript

Cadena

objeto

(Java, Python…)



JSON

Formato de intercambio de datos abierto

- También se usa, por ejemplo, como formato de ficheros de configuración

Basado en la notación para Objects y Arrays de JS (JavaScript Object Notation)

Usa una sintaxis muy comprensible para humanos

Los objetos se representan mediante un conjunto de pares atributo-valor

También permite representar arrays de objetos o valores

Independiente del lenguaje de programación

Estándar ECMA desde 2013 (ECMA-404) e ISO desde 2017 (STD-90, ISO/IEC 21778:2017)

Especificado por primera vez por Douglas Crockford

Tipos de datos

Sintaxis

Especificación: https://www.json.org/json-en.html

```
{
"employees": [
"id": 1,
"name": "Juan Ruiz",
"salary": 1500
},
"id": 2,
"name": "Ana Ferrer",
"salary": 1700
}
```

Otros elementos de la sintaxis

Se permite el uso de espacios en blanco

- Si están alrededor de los elementos, se ignora
- Si están dentro de una cadena, se consideran

No se permiten los comentarios

Tipo MIME: application/json

La indentación no es necesaria, pero permite una mejor lectura

- Para mejorar la eficiencia en la transmisión, se suele eliminar la indentación al enviar JSON por la red
- Se puede volver a aplicar indentación usando herramientas(p.ej. https://codebeautify.org/jsonviewer)

Alternativas

Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

Comunicación síncrona

Cuando hay comunicación síncrona entre dos entes A y B, si A envía un mensaje a B, deja de actuar hasta que B responde

- También llamada “bloqueante”, porque A se bloquea esperando

Comunicación asíncrona

Cuando hay comunicación asíncrona entre dos entes A y B, si A envía un mensaje a B, puede seguir actuando (incluso enviando otros mensajes a B o otros entes). B responderá cuando pueda y enviará un mensaje de vuelta a A

- También llamada “no bloqueante”

Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

AJAX

AJAX (Asynchronous JavaScript And XML): Término acuñado en 2005 para describir una nueva forma de abordar las peticiones HTTP al servidor para hacer actualizaciones parciales de la página en cliente, sin tener que hacer una recarga completa.

Inicialmente, se usaba XML como formato de comunicación, pero hoy en día lo habitual es usar JSON

El núcleo de AJAX es la API XMLHttpRequest (XHR) de JS

- Documentación: https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest

Algunas implementaciones

- Nativas: XHR, fetch
- Librerías externas: jQuery, axios

XMLHttpRequest (XHR)

XHR puede usarse para hacer peticiones síncronas o asíncronas

- Parámetro opcional “async” del método open()
- Por defecto, asíncronas
- Se recomienda usar peticiones asíncronas, que ofrecen una experiencia más fluida al usuario

Se basa en el uso de funciones callback

- Se asocian funciones que se ejecutarán cuando ocurran ciertos eventos sobre la petición, como que se haya respondido o que haya ocurrido un error.

Ventajas

- Soporte nativo
- Permite gran control a bajo nivel sobre la petición

Desventajas

Fetch

API nativa de JavaScript basada en promesas

- Documentación: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

La respuesta a una petición hecha con fetch es una promesa

- Dado que la comunicación es asíncrona, no se pueden devolver los datos inmediatamente, solo la promesa de que cuando lleguen los datos, se ejecutará un cierto fragmento de código

Cuando se resuelve la promesa, se puede consultar el objeto Response y sus propiedades: Status, statusText, Headers, …

Ventajas

- Nativo, interfaz más a alto nivel que XHR

Desventajas

- La compatibilidad con navegadores no es del 100% aún
- La lógica para obtener los datos de una respuesta requiere encadenar como mínimo dos promesas y es menos limpia que otras alternativas

jQuery

La librería jQuery incluye (entre otras muchas cosas) diversos métodos para realizar peticiones AJAX

- Documentación: https://api.jquery.com/category/ajax/

Basada en XMLHttpRequest (XHR)

Ventajas

- Respecto a XHR: Facilita la escritura de código para realizar peticiones AJAX
- Respecto a fetch: Al ser parte de jQuery, es cross-browser

Desventajas

- Librería externa
- Enviar formularios mediante objetos FormData nativos requiere una configuración poco intuitiva
- La librería jQuery está cayendo cada vez más en desuso al tener los navegadores una API cada vez más estándar

Axios

Librería basada en XMLHttpRequest (la usa internamente), pero usa funciones asíncronas en vez de callbacks.

Ventajas

- Intercepta request y response: permite examinar y modificar las peticiones y respuestas HTTP (por ejemplo, para login o autenticación)
- Permite cancelar requests
- Múltiples librerías de terceros para ampliar la funcionalidad
- Compatibilidad con muchos navegadores, incluso antiguos
- Simplifica la lógica de gestión de respuestas y errores
- Muy popular actualmente, disponible también para su uso en backend con NodeJS

Desventajas

- Librería externa

Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

Peticiones GET

Necesitamos importar axios:

```
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

Esquema base:

```
await axios.get/post/put/delete(url, requestOptions)
```

Petición GET (I)

Ejemplo: consultar todas las fotos

```
try {
let response = await axios.get("/api/v1/photos");
let photos = response.data;
console.log(photos);
} catch (err) {
console.error(err);
}
```

Petición GET (I)

El objeto response

```
try {
let response = await axios.get("/api/v1/photos");
let photos = response.data;
console.log(photos);
} catch (err) {
console.error(err);
}
```

data: Datos devueltos por el servidor, ya deserializados

status: Código HTTP devuelto

statusText: Mensaje textual del código HTTP devuelto

headers: Cabeceras de la respuesta HTTP del servidor

ferencia a la petición axios asociada a esta respuesta

Petición GET (II)

Ejemplo: consultar una URL que no existe

```
try {
let response = await axios.get("/api/v1/9999999999");
let photos = response.data;
console.log(photos);
} catch (err) {
console.error(err);
}
```

Petición GET (II)

El objeto error

```
try {
let response = await axios.get("/api/v1/9999999999");
let photos = response.data;
console.log(photos);
} catch (err) {
console.error(err);
}
```

description: Descripción textual del error

request: Referencia a la petición axios que provoca el error

response: Respuesta del servidor, con los mismos atributos que una respuesta normal (data, status…)

petición que devuelva un código HTTP de error (4XX, 5XX) eleva una

Petición GET (III)

En cualquier petición axios se puede pasar un parámetro adicional para configurar la petición (cabeceras HTTP, timeout…)

```
let options = {
headers: {
Token: "ABCDE123",
},
timeout: 5000, // milliseconds
};
try {
let response = await axios.get("/api/v1/photos", options);
let photos = response.data;
log(photos);
```

Peticiones POST/PUT

Esquema base:

```
await axios.post/put(url, body, requestOptions)
```

Petición POST/PUT (I)

En las peticiones POST y PUT se puede pasar un objeto JS, que se serializará a JSON y se enviará como cuerpo de la petición

```
let photo = {
url: "/images/example.jpg",
date: "2020-04-20 13:37:00",
title: "New photo",
description: "A new photo",
visibility: "Public",
userId: 2
};
try {
se = await axios.post("/api/v1/photos", photo);
```

Petición POST/PUT (II)

Si los datos provienen de un formulario, se puede enviar un objeto FormData construido a partir del <form> para evitar tener que construir el cuerpo de la petición manualmente:

```
let form = document.getElementById("photo-form");
let formData = new FormData(form);
try {
let response = await axios.post("/api/v1/photos", formData);
console.log(response.data);
} catch (err) {
console.log(err);
}
```

Las peticiones PUT son equivalentes usando el método axios.put()

Ejemplo POST: Registro de usuario

<form id="register-form">

First name:

First name

Last name:

Last name

Telephone:

Telephone number

Email:

@ Email

Profile picture:

URL to your profile picture

Username:

Username

Password:

Password

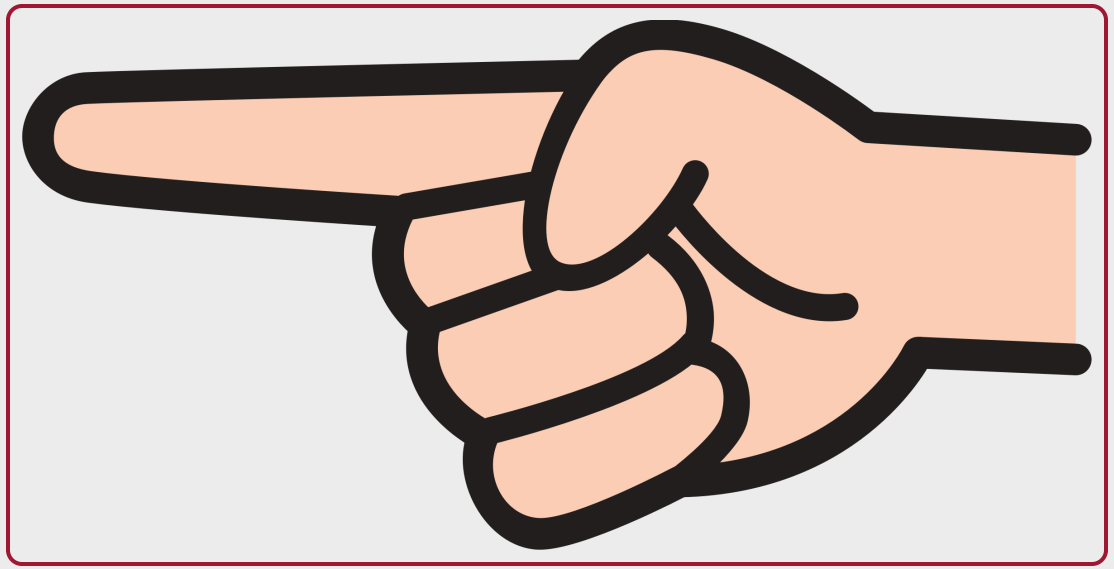
Repeat your password:

Repeat your password

Ejemplo POST: Registro de usuario

```
<form id="register-form">
let form = document.getElementById("register-form");
let formData = new FormData(form);
try {
let response = await axios.post("/api/v1/register", formData);
console.log(response.data);
} catch (err) {
console.log(err);
}
```

Ejemplo POST: Registro de usuario



Se devuelve el usuario creado y el token de sesión

Peticiones DELETE

Esquema base:

```
await axios.delete(url, requestOptions)
```

Petición DELETE (I)

Sólo es necesario indicar la URL:

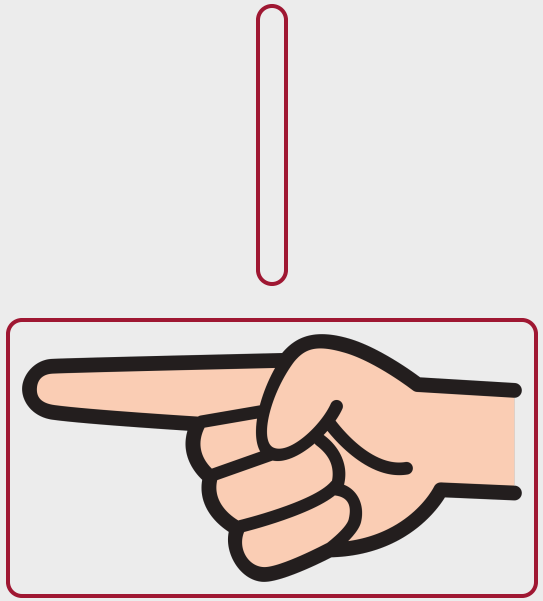
```
try {
let response = await axios.delete("/api/v1/photos/3");
console.log(response.data);
} catch (err) {
console.log(err);
}
```

Petición DELETE (II)

Al igual que el resto de métodos, si el servidor devuelve un error, se ejecuta el "catch":

```
await axios.delete("/api/v1/photos/3")
```

Foto con comentarios



Contenidos

REST

Formatos de comunicación (JSON)

Recordatorio: modelo de comunicación asíncrona

Implementación de llamadas asíncronas REST

Ejemplos

Módulos de comunicación con API

Módulos API

Módulos JS encargados de encapsular las peticiones a una API REST externa

Permiten aislar al resto de la aplicación de la implementación concreta para realizar peticiones AJAX proporcionando una interfaz común estándar

Acceso programático a las consultas, modificaciones, inserciones y borrado de datos

Centralizan la configuración de elementos comunes como la URL base o las cabeceras que deben enviarse en todas las peticiones (por ejemplo, tokens de sesión)

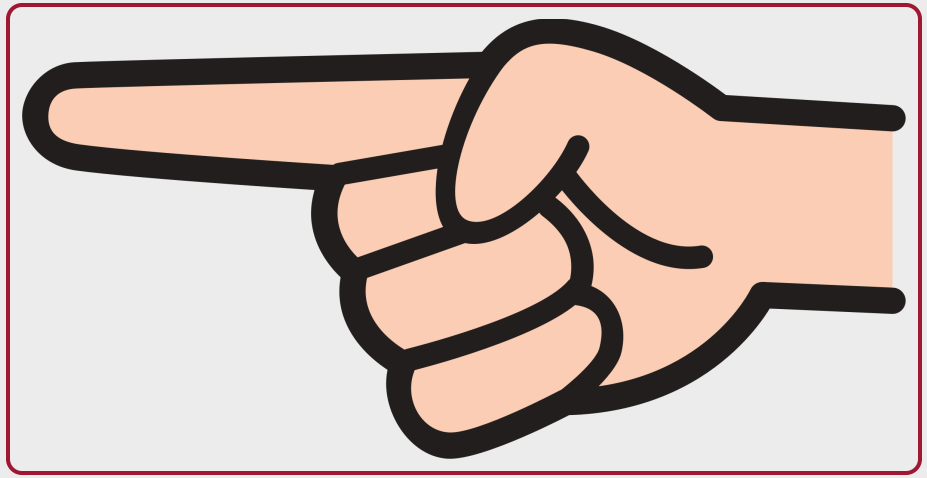
Estructura del proyecto

Organización del código

- Nosotros seguiremos la siguiente estructura

```
js
├── api
├── libs
├── renderers
├── utils
├── validators
├── vista_1.js
├── vista_2.js
⋮
```

UNIVERSIDAD D SEVILLA



Módulo de configuración común de API

/js/api/common.js

```
"use strict";
import { sessionManager } from "/js/utils/session.js";
const BASE_URL = "/api/v1";
const requestOptions = {
headers: { Token: sessionManager.getToken() },
};
export { BASE_URL, requestOptions };
```

Se define la URL base y la configuración común para todas las peticiones

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function (formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Módulo de acceso a API para la entidad Fotos

```
import { BASE_URL, requestOptions } from './common.js';
const photosAPI = {
getAll: async function() {
let response = await axios.get( ${BASE_URL}/photos , requestOptions);
return response.data;
},
create: async function(formData) {
let response = await axios.post( ${BASE_URL}/photos , formData, requestOptions);
update: async function(formData, photoId) {
let response = await axios.put( ${BASE_URL}/photos/${photoId} , formData,
requestOptions);
};
export { photosAPI };
```

Recordatorio

Los módulos de API se pueden generar automáticamente con el comando silence createapi

Ejemplo de uso: API + renderizadores

```
import { photosAPI } from ’/js/api/photos.js';import { galleryRenderer } from
'/js/renderers/gallery.js';
import { messageRenderer } from '/js/renderers/messages.js';
async function loadPhotos() {
try {
let galleryContainer = document.getElementById("gallery-container");
let photos = await photosAPI.getAll();
let cardGallery = galleryRenderer.asCardGallery(photos);
galleryContainer.appendChild(cardGallery);
} catch (err) {
messageRenderer.showErrorAsAlert("Error showing photos", err);
}
```

Ejemplo de uso: API + renderizadores

```
import { photosAPI } from ’/js/api/photos.js';import { galleryRenderer } from
'/js/renderers/gallery.js';
import { messageRenderer } from '/js/renderers/messages.js';
async function loadPhotos() {
try {
let galleryContainer = document.getElementById("gallery-container");
let photos = await photosAPI.getAll();
let cardGallery = galleryRenderer.asCardGallery(photos);
galleryContainer.appendChild(cardGallery);
} catch (err) {
messageRenderer.showErrorAsAlert("Error showing photos", err);
}
```

Uso de JavaScript (III) Llamadas AJAX mediante REST y JSON

Departamento de Lenguajes y Sistemas Informáticos

Universidad de Sevilla

Daniel Ayala, Carlos Arévalo, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz